

# Чат с LLM: разработка ERC-4626 Yield Vault

## Использование генеративных моделей

- **Модель:** ChatGPT-4o (OpenAI, 2025).
- **Доля кода, сгенерированного моделью:** ~20–25% (ниже 30%).
- **Где использовалось:**
  - черновая генерация шаблонного кода Solidity (заготовки контрактов, интерфейсы адаптеров);
  - помощь с оформлением тестов Foundry и настройкой fuzz/invariant-конфигурации;
  - проверка типовых практик работы с OpenZeppelin, ERC-4626, EIP-712;
  - помощь в написании Python-агента (структура scoring-модуля, ЕМА-логика);
  - проверка формул нормализации ставок (Aave RAY, Compound per-second).
- **Основные источники, на которые опирался:**
  - документация OpenZeppelin: <https://docs.openzeppelin.com/contracts/5.x/>
  - стандарт ERC-4626: <https://eips.ethereum.org/EIPS/eip-4626>
  - стандарт EIP-712: <https://eips.ethereum.org/EIPS/eip-712>
  - документация Aave V3: <https://docs.aave.com/developers/>
  - документация Compound V3 (Comet): <https://docs.compound.finance/>
  - документация Foundry: <https://book.getfoundry.sh/>
  - документация Chainlink Automation: <https://docs.chain.link/chainlink-automation>
  - материалы курса и формулировка задания.
- **Пример промпта:** «Сгенерируй черновой шаблон ERC-4626 vault с UUPS proxy и Reentrancy-Guard. Нужны функции *deposit*, *withdraw*, *rebalance*. Не меняй архитектуру; финальные правки и проверку безопасности оставь за мной.»
- **Опыт использования / ограничения:** LLM ускорила рутинные части (boilerplate-код, конфигурация тестов), но потребовались ручные правки: проверка порядка наследования OpenZeppelin, корректность `_decimalsOffset` для inflation attack protection, правильная реализация EIP-712 domain separator, настройка Foundry invariant handler.
- **Ответственность за решения:** Все ключевые решения (архитектура vault, формулы MCDM-модели, выбор весов скоринга, дизайн системы безопасности и интерпретация результатов тестирования) принимались мной; модель использовалась как ассистент.

## Фрагменты диалога

Ниже приведены ключевые фрагменты переписки с LLM, демонстрирующие процесс разработки.

---

### Диалог 1: Архитектура vault

#### Пользователь:

Мне нужно спроектировать DeFi vault, который автоматически перекладывает USDC между Aave V3 и Compound V3 для максимизации доходности. Vault должен быть ERC-4626 совместимым, upgradeable (UUPS proxy), и иметь защиту от inflation attack. Какую архитектуру ты бы предложил?

#### ChatGPT:

Для вашего случая рекомендую трехуровневую архитектуру:

##### 1. **AlVault.sol** – основной контракт vault:

- Наследует ERC4626Upgradeable, UUPSUpgradeable, ReentrancyGuardUpgradeable
- Хранит shares пользователей, управляет deposit/withdraw
- Для защиты от inflation attack используйте `_decimalsOffset() = 6`, это добавит  $10^6$  виртуальных shares

##### 2. **StrategyManager.sol** – логика принятия решений:

- Проверяет подписи агента (EIP-712)
- Хранит nonce, cooldown, параметры стратегии
- Отделен от vault для возможности апгрейда логики отдельно

##### 3. **IProtocolAdapter** – интерфейс для протоколов:

- AaveV3Adapter – обертка над Aave V3 Pool
- CompoundV3Adapter – обертка над Compound Comet
- Новые протоколы добавляются без изменения vault

Для inflation attack: стандартная защита через virtual shares offset. OpenZeppelin ERC4626 уже поддерживает `_decimalsOffset`, просто переопределите его на 6.

#### Пользователь:

Хорошо. А как быть если агент упадет или будет недоступен? Vault не должен оставаться без управления.

#### ChatGPT:

Рекомендую добавить fallback через Chainlink Automation:

- Реализуйте интерфейс `AutomationCompatibleInterface` с функциями `checkUpkeep` и `performUpkeep`

- В checkUpkeeper проверяйте: прошло ли больше 6 часов с последней ребалансировки И есть ли экономическая выгода от переключения
- Формула выгоды:  $\text{dailyBenefit} = \text{TVL} * \text{deltaAPY} / 365 \text{ days}$
- Ребалансировка через Chainlink происходит только если  $\text{benefitOverPeriod} > \text{estimatedGasCost}$

Так у вас будет два пути: основной (агент с MCDM) и fallback (Chainlink с простой cost-benefit проверкой).

## Диалог 2: Формулы MCDM-модели

### Пользователь:

Я не хочу просто сравнивать APY между протоколами – это наивный подход. Нужна многофакторная модель. Какие факторы стоит учитывать для скоринга lending-протоколов?

### ChatGPT:

Для MCDM (Multi-Criteria Decision Making) модели в контексте DeFi lending рекомендую 4 фактора:

- 1. APY (вес 40%)** – основной сигнал доходности. Нормализуйте к диапазону [0, 1] через деление на максимальный ожидаемый APY (например, 20%).
- 2. Risk (вес 25%)** – обратная функция от utilization rate. Высокая утилизация (>90%) означает риск скачка ставок и проблем с ликвидностью. Формула:  $f_{\text{risk}} = 1 - \text{normalize}(\text{utilization}, 0, 1)$ .
- 3. Cost (вес 20%)** – стоимость газа для ребалансировки относительно TVL. Формула:  $f_{\text{cost}} = 1 - \text{normalize}(\text{gasCost}/\text{TVL}, 0, 0.01)$ .
- 4. Stability (вес 15%)** – волатильность TVL протокола. Резкое падение TVL может означать проблемы. Формула:  $f_{\text{stability}} = 1 - \text{normalize}(\text{abs}(\text{deltaTVL}), 0, 0.30)$ .

Итоговый скор:  $\text{Score}_i = 0.40 \cdot f_{\text{APY}} + 0.25 \cdot f_{\text{Risk}} + 0.20 \cdot f_{\text{Cost}} + 0.15 \cdot f_{\text{Stability}}$

Ребалансировку выполняйте только если разница скоров  $\geq 0.05$ , чтобы избежать избыточных переключений.

### Пользователь:

Веса я подберу сам по результатам тестов. А как сглаживать ставки, чтобы не реагировать на краткосрочные манипуляции?

### ChatGPT:

Используйте экспоненциальное скользящее среднее (EMA):

$$S_t = \alpha \cdot R_t + (1 - \alpha) \cdot S_{t-1}$$

где  $\alpha = 0.3$  (30% веса на новое наблюдение). Это хороший баланс между отзывчивостью и стабильностью.

Дополнительно добавьте **jump guard**: если ставка изменилась больше чем на 5% (500 bps) за один период, пропустите обновление ЕМА. Это защита от flash loan атак, когда атакующий может искусственно зависить ставку на один блок.

Реализация на Solidity: храните smoothedRate в mapping по адресу протокола, обновляйте при каждом вызове updateRate.

### Диалог 3: Шаблон ERC-4626 vault

#### Пользователь:

Сгенерируй черновой шаблон AIVault.sol с наследованием от OpenZeppelin 5.x. Нужны: ERC4626, UUPS, ReentrancyGuard, Ownable. Функцию rebalance пока оставь пустой, я допишу сам.

#### ChatGPT:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts-upgradeable/token/ERC20/
    extensions/ERC4626Upgradeable.sol";
import "@openzeppelin/contracts-upgradeable/proxy/utils/
    UUPSUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/utils/
    ReentrancyGuardUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/access/
    OwnableUpgradeable.sol";

contract AIVault is
    ERC4626Upgradeable,
    UUPSUpgradeable,
    ReentrancyGuardUpgradeable,
    OwnableUpgradeable
{
    IStrategyManager public strategyManager;
    uint256 public lastRebalance;
    uint256 public constant COOLDOWN = 1 hours;

    function initialize(
        IERC20 asset_,
        address strategyManager_,
        address owner_
    ) external initializer {
        __ERC4626_init(asset_);
        __ERC20_init("AI Yield USDC", "aiUSDC");
        __UUPSUpgradeable_init();
        __ReentrancyGuard_init();
        __Ownable_init(owner_);
        strategyManager =
            IStrategyManager(strategyManager_);
    }

    function _decimalsOffset()
```

```

        internal pure override returns (uint8)
    {
        return 6; // inflation attack protection
    }

    function rebalance(bytes calldata signature)
        external nonReentrant
    {
        // TODO: verify signature, check cooldown,
        // execute via adapter
    }

    function _authorizeUpgrade(address)
        internal override onlyOwner {}
}

```

Порядок наследования важен: ERC4626 первым (из-за linearization), затем UUPS, Reentrancy-Guard, Ownable. Проверьте что `_decimalsOffset = 6` дает  $10^6$  виртуальных shares.

#### Пользователь:

Порядок наследования проверил, все корректно. Функцию `rebalance` я реализовал сам с EIP-712 верификацией и `cooldown`-проверкой. Спасибо за шаблон.

## Диалог 4: Нормализация ставок

#### Пользователь:

Aave V3 хранит ставки в формате RAY ( $10^{27}$ ), а Compound V3 – per-second rate. Мне нужно привести оба к единому годовому формату в масштабе  $1e18$ . Как правильно сделать конвертацию?

#### ChatGPT:

Для Aave V3:

```

// liquidityRate is in RAY (1e27)
// Target: APY in 1e18 scale
// Simply divide by 1e9:
uint256 apyE18 = liquidityRate / 1e9;

```

Для Compound V3 (Comet):

```

// getSupplyRate returns per-second rate
// in 1e18 scale
// Multiply by seconds per year:
uint256 SECONDS_PER_YEAR = 31_557_600;
// (365.25 days, accounts for leap years)
uint256 apyE18 = supplyRate * SECONDS_PER_YEAR;

```

Важно: используйте 31,557,600 секунд (365.25 дней), а не 31,536,000 (365 дней), чтобы корректно учесть високосные годы. Разница небольшая, но для точности формул это важно.

Вынесите обе конвертации в отдельную библиотеку `RateMath.sol` для переиспользования.

## Диалог 5: Настройка Foundry invariant-тестов

### Пользователь:

Я написал unit-тесты и интеграционные тесты. Теперь хочу добавить invariant-тесты через Foundry. Нужно проверить 6 инвариантов: solvency, accounting consistency, share price monotonicity, round-trip conversions, supply consistency, rounding direction. Как настроить handler?

### ChatGPT:

Создайте handler-контракт, который Foundry будет вызывать случайным образом:

```
contract VaultHandler is Test {
    AIVault vault;
    address[] actors;

    function deposit(uint256 actorSeed,
        uint256 amount) external {
        address actor = actors[
            actorSeed % actors.length];
        amount = bound(amount, 1e6, 1_000_000e6);
        vm.startPrank(actor);
        usdc.approve(address(vault), amount);
        vault.deposit(amount, actor);
        vm.stopPrank();
    }

    function withdraw(uint256 actorSeed,
        uint256 amount) external {
        // similar with bound()
    }

    function rebalance() external {
        // simulate agent rebalance
    }
}
```

В foundry.toml:

```
[invariant]
runs = 128
depth = 600
```

Это даст 128 прогонов × 600 вызовов = 76,800 случайных действий. Каждый инвариант проверяется после каждого вызова.

Для проверки solvency: `totalAssets() >= sum(maxWithdraw(actor))` для всех акторов.

### Пользователь:

Настроил handler по этому шаблону, добавил все 6 инвариантов. После 76,800 вызовов – 0 нарушений. Все инварианты выполняются.

---

## Диалог 6: Python scoring-модуль

### Пользователь:

Помоги написать черновик `scoring.py` для Python-агента. Нужна функция `score_protocols`, которая принимает список протоколов с метриками и возвращает скоры по MCDM-модели. Беса: APY 0.40, Risk 0.25, Cost 0.20, Stability 0.15.

### ChatGPT:

```
WEIGHTS = {
    "apy": 0.40,
    "risk": 0.25,
    "cost": 0.20,
    "stability": 0.15,
}

def normalize(value, min_val, max_val):
    clamped = max(min_val, min(value, max_val))
    return clamped / (max_val - min_val)

def score_protocol(metrics, gas_price):
    f_apy = normalize(metrics["apy"], 0, 0.20)
    f_risk = 1 - normalize(
        metrics["utilization"], 0, 1)
    f_cost = 1 - normalize(
        gas_price / metrics["tv1"], 0, 0.01)
    f_stability = 1 - normalize(
        abs(metrics["tv1_delta"]), 0, 0.30)

    return (WEIGHTS["apy"] * f_apy
            + WEIGHTS["risk"] * f_risk
            + WEIGHTS["cost"] * f_cost
            + WEIGHTS["stability"] * f_stability)

def score_protocols(protocols, gas_price):
    scores = {}
    for name, metrics in protocols.items():
        scores[name] = score_protocol(
            metrics, gas_price)
    best = max(scores, key=scores.get)
    return scores, best
```

Рекомендую добавить порог: ребалансировка только если `scores[best] - scores[current] >= 0.05`.

### Пользователь:

Взял за основу, доработал: добавил ЕМА-сглаживание ставок перед скорингом, логирование, и проверку порога 0.05. Также написал 20 unit-тестов на `pytest` для граничных случаев.

---

## Диалог 7: EIP-712 подпись в Python

### Пользователь:

Агент должен подписывать решение о ребалансировке через EIP-712. Как реализовать подпись

typed data в Python с web3.py?

**ChatGPT:**

Используйте eth\_account (входит в web3.py):

```
from eth_account import Account
from eth_account.messages import encode_typed_data
```

```
domain = {
    "name": "AIVault",
    "version": "1",
    "chainId": 11155111,
    "verifyingContract": vault_address,
}
```

```
types = {
    "Rebalance": [
        {"name": "targetAdapter", "type": "address"},
        {"name": "nonce", "type": "uint256"},
        {"name": "timestamp", "type": "uint256"},
    ]
}
```

```
message = {
    "targetAdapter": target_adapter,
    "nonce": current_nonce,
    "timestamp": int(time.time()),
}
```

```
signable = encode_typed_data(
    domain, types, message)
signed = Account.sign_message(
    signable, private_key)
signature = signed.signature.hex()
```

На стороне Solidity используйте ECDSA.recover(digest, signature) от OpenZeppelin для верификации.

---

## Итог

LLM использовалась как ассистент для ускорения рутинных задач. Все архитектурные решения, формулы MCDM-модели, дизайн системы безопасности, финальная реализация и тестирование выполнены самостоятельно. Сгенерированный код составляет не более 20–25% от общего объема и был существенно переработан.